Roteiro BackEnd – Aplicação WEB

1. Requisitos básicos
    a. Banco de Dados MySQL e MySQL WorkBench

    Nome do banco de Dados: **bd_agenda**

```
CREATE TABLE agenda
(
        codigo      int           auto_increment primary key,
        nome        varchar(50)   not null,
        endereco    varchar(50)   not null,
        telefone    char(11)      not null,
        email       varchar(60)
);
```

    Para utilizar o banco de forma local – usuário root com senha

    ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'mysql4@11' ;

    Ou

    Para utilizar o banco de forma local – usuário root sem senha

```
ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password ;
```

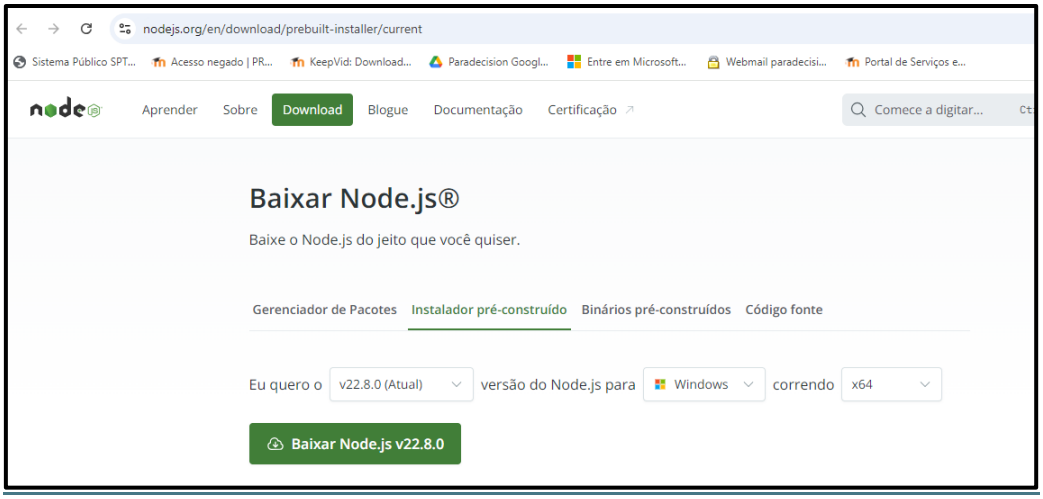
    b. Construindo o Back-end: Node

    Node (ou formalmente *Node.js*) é um ambiente em tempo de execução open-source (código aberto) e multiplataforma que permite aos desenvolvedores criarem todo tipo de aplicativos e ferramentas do lado servidor (backend) em JavaScript. Node é

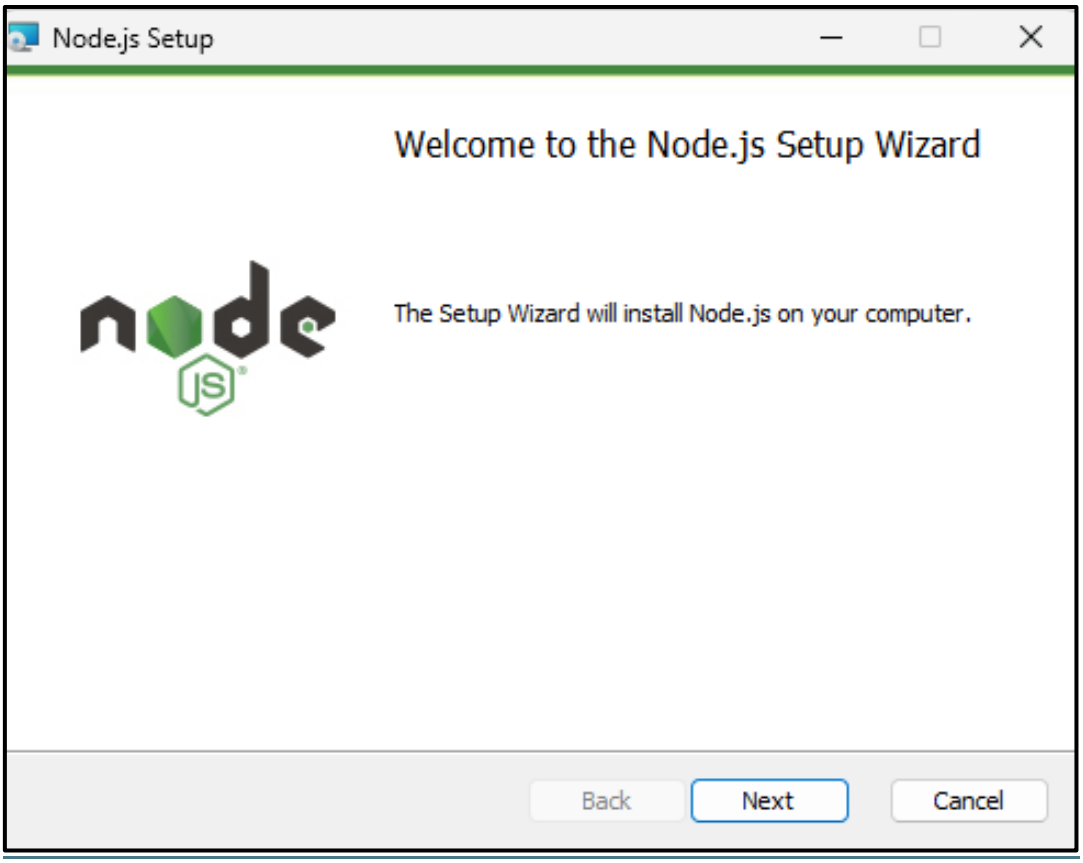
usado fora do contexto de um navegador (ou seja, executado diretamente no computador ou no servidor). Como tal, o ambiente omite APIs JavaScript específicas do navegador e adiciona suporte para APIs de sistema operacional mais tradicionais, incluindo bibliotecas de sistemas HTTP e arquivos.

- Performance excelente. Node foi projetado para otimizar a taxa de transferência e a escalabilidade em aplicações web. É uma ótima combinação para resolver muitos problemas comuns no desenvolvimento da web (por exemplo, aplicações em tempo real).
- O Gerenciador de Pacotes do Node (NPM, na sigla em inglês) provê acesso a centenas de milhares de pacotes reutilizáveis. NPM possui uma coleção de dependências e pode ser usado para automatizar a cadeia de ferramentas de compilação.

- Instalação Node.js
- https://nodejs.org/en



Após o download – Iniciar a instalação



- Será exibido as duas versões principais para download
- Escolher a versão: 22.8.0 LTS
- Baixar o arquivo: node-v22.8.0-x64.msi
- Para Verificar a instalação
- No prompt do DOS ou no terminal do VSCode, digite:

  node -v

  o retorno será a versão do node: v22.8.0
- Após a instalação estará disponível o comando npm
- Opcional: Pode-se instalar o gerenciar do de pacotes yarn ou utilizar o npm:

  No prompt do DOS ou no terminal do VSCode, digite:

  npm install - -global yarn
- Verificar a instalação do yarn
- No prompt do DOS ou no terminal do VSCode, digite:

  yarn –version

o retorno será a versão do yarn: 1.22.22

2. Construindo o código do BackEnd no VSCode:
   a) Em qualquer diretório do seu **computador criar uma pasta de nome AgendaFullStack**. Servirá como repositório para o nosso projeto.
   b) No VSCode: File – Open Folder; Selecione a pasta recém criada.
   c) Sobre o nome da pasta/projeto incorporado ao VSCode **crie a pasta: backend_node** (será o nosso servidor backend). Utilize a opção New Folder
   d) Abra um novo terminal no VSCode: CTRL + J
   e) Acessar a pasta backend_node utilizando o comando:

      cd backend_node

   f) Criar um projeto node:

      npm init -y

   g) Será criado o arquivo **package.json**
   h) Instalar as bibliotecas: express, nodemon, mysql e cors

a. express

Express é um framework Node e a biblioteca subjacente para uma série de outros frameworks do Node. O Express oferece soluções para:

- Gerenciar requisições de diferentes verbos HTTP em diferentes URLs.
- Integrar "view engines" para inserir dados nos templates.
- Definir as configurações comuns da aplicação web, como a porta a ser usada para conexão e a localização dos modelos que são usados para renderizar a resposta.
- Adicionar novos processos de requisição por meio de "middleware" em qualquer ponto da "fila" de requisições.

b. nodemon

O Nodemon é uma ferramenta que permite reiniciar automaticamente o servidor Node.js sempre que alterações

nos arquivos da aplicação são detectadas. Essa funcionalidade agiliza o processo de desenvolvimento, eliminando a necessidade de reinicializações manuais do servidor.

c. mysql

O Node.js oferece vários pacotes para interagir com bancos de dados MySQL. Um dos pacotes mais populares é o mysql .O pacote facilita a execução de consultas SQL, permitindo acesso ao MySQL sem problemas.

d. cors

Permite relaxar a segurança aplicada a uma API. Neste caso, o servidor irá trabalhar de forma local, o cors evitará conflitos

No terminal do VSCode digite:

npm add express
npm add nodemon
npm add mysql
npm add cors

ou

yarn add express nodemon mysql cors

i) Configurando o arquivo package.json, criado anteriormente

```
Incluir/Alterar as linhas amarelas
Remover linha em vermelho
```

A linha **“type”: “module”** irá permitir a inclusão de bibliotecas e módulos ao nosso projeto

```
{
  "name": "backend_node",
  "version": "1.0.0",
  "main": "index.js",
  "type": "module",
 "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
    "start": "nodemon index.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.18.2",
    "mysql": "^2.18.1",
    "nodemon": "^3.0.3"
  },
  "devDependencies": {},
  "description": ""
}
```

j) Dentro da pasta backend_node criar o arquivo **db.js**

// Incorpora o módulo mysql ao nosso código; associando o módulo
// a mysql, que será utilizada para criar a conexão com o banco de
// dados
// host indica o IP do servidor de banco de dados, neste caso,
// localhost faz uma referência ao IP local: 127.0.0.1
// user indica o usuário utiliza para autenticar a conexão com o
// banco de dados
// passwd indica a senha do usuário de conexão, neste caso, o
// usuário não possui senha
// database indica o nome do banco de dados utilizado pela
// aplicação

```
import mysql from "mysql";
```

```
export const db = mysql.createConnection({
    host: "localhost",
    user: "root",
    password: "",
    database: "bd_agenda",
});
```

k) Dentro da pasta backend_node criar o arquivo: index.js

```
import express from "express";
import agendaRoutes from "./routes/agendas.js";
import cors from "cors";
//Para utilizar o modo json para alterações como
// Post e Put
const app = express();
app.use(express.json());
app.use(cors());
app.use("/", agendaRoutes);
//Escutar a porta 8800
app.listen(8800);
```

l) Dentro da pasta backend_node criar, a pasta **routes**

m) No diretório routes, criar o arquivo **agendas.js**

```
import express from "express";
import {
    getAgendas,
    addAgenda,
    updateAgenda,
    deleteAgenda
} from "../controllers/agenda.js";

const router = express.Router();

router.get("/", getAgendas);

router.post("/", addAgenda);

router.put("/:codigo", updateAgenda);
```

```
router.delete("/:codigo", deleteAgenda);

export default router;
```

n) Dentro da pasta backend_node criar, a pasta **controllers**

o) Criar o arquivo **agenda.js** dentro de controllers

```
import { db } from "../db.js";

//No caso do get não é necessário incluir req de requisição
export const getAgendas = (_, res) => {
    const q = "SELECT * FROM tb_agenda ORDER BY nome";

    db.query(q, (err, data) => {
        if (err) return res.json(err);

        return res.status(200).json(data);
    });
};

export const addAgenda = (req, res) => {
    const q = "INSERT INTO tb_agenda (`nome`,
`endereco`,`telefone`,`email`) VALUES(?)";

    const values = [
        req.body.nome,
        req.body.endereco,
        req.body.telefone,
        req.body.email,
    ];

    db.query(q, [values], (err) => {
        if (err) return res.json(err);

       return res.status(200).json("Registro criado com
sucesso");
    });
};

export const updateAgenda = (req, res) => {
    const q = "UPDATE tb_agenda SET `nome` = ?, `endereco` =
? ,`telefone` = ?,`email` = ? WHERE `codigo` = ?";

    const values = [
        req.body.nome,
        req.body.endereco,
```

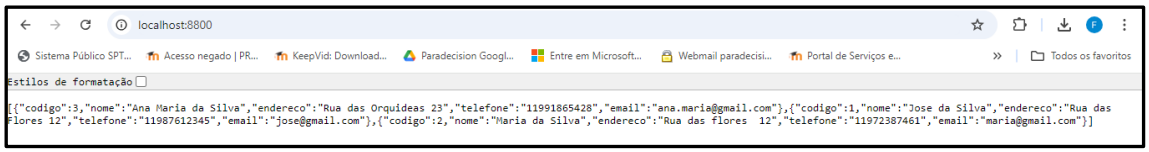
```
        req.body.telefone,
        req.body.email,
    ];

    db.query(q, [...values, req.params.codigo], (err) => {
        if (err) return res.jason(err);

        return res.status(200).json("Registro atualizado com
sucesso");
    });
};

export const deleteAgenda = (req, res) => {
    const q = "DELETE FROM tb_agenda WHERE `codigo` = ?";

    db.query(q, [req.params.codigo], (err) => {
        if (err) return res.json(err);

        return res.status(200).json("Registro deletado com
sucesso");
    });
};
```

p) Start o servidor backend para testar – No terminal, dentro da backend_node

cd backend_node

npm start

ou

yarn start

No navegador: digitar : localhost:8800